

The ENSAI logo consists of the text "ENSAI" in a bold, sans-serif font, followed by a stylized circular graphic made of dashed blue lines.

École nationale
de la statistique
et de l'analyse
de l'information

Création d'un classifieur permettant la codification automatique de libellés de produit dans la nomenclature COICOP

Rapport de stage

KACER MATHIS

STAGE 2A FONCTIONNAIRE

Juin – Août 2026

Référent pédagogique :

ANYSIA THOMAS

anysia.thomas@insee.fr

Tuteurs de projet :

JULIEN PRAMIL

julien.pramil@insee.fr

Insee 
Mesurer pour comprendre

Table des matières

1	Le stage : lieu, sujet et objectif	2
2	Les modèles de classification automatique actuels	2
2.1	Prétraitement et règles (regex)	2
2.2	Les quatre classifieurs de base	3
2.3	L'arbitrage final par LLM-as-a-judge	3
3	Démarche et avancées du stage	3
3.1	Une baseline low-cost : vote majoritaire et arbitrage TTC	5
3.2	Détecter les erreurs de la baseline par Random Forest	6
3.3	Analyse des erreurs manquées et compromis précision/rappel	7
3.4	Robustesse : généralisation à un nouveau run et correctif de convention . .	8
3.5	Une piste interprétable : SIRUS	9
3.6	Bilan comparatif des approches	9
4	Conclusion et perspectives	11

1 Le stage : lieu, sujet et objectif

Ce stage s'est déroulé à la direction générale de l'INSEE, à Montrouge, au sein du **SSP Lab**, l'unité d'innovation en data science du Service Statistique Public, rattachée à la Direction de la Méthodologie et de la Coordination Statistique et Internationale (DMCSI).

Son sujet, en tant que stage 2A fonctionnaire, était la **création d'un classifieur permettant la codification automatique de libellés de produits dans la nomenclature COICOP**, pour l'enquête **Budget de Famille (BdF)**. Cette enquête décrit l'ensemble des dépenses et des ressources des ménages résidant en France ; chaque dépense déclarée — un libellé textuel brut, issu d'un carnet, d'un ticket de caisse ou d'une saisie en application — doit être associée à un poste de la nomenclature **COICOP** (*Classification of Individual Consumption According to Purpose*) pour produire des statistiques de consommation comparables au niveau international, transmises à Eurostat. Codée manuellement, cette association est coûteuse en temps et difficilement soutenable à l'échelle de l'enquête : l'enjeu du stage était donc de contribuer à son automatisation.

Cette codification automatique de la BdF n'est pas un projet isolé : la classification automatique de libellés textuels dans une nomenclature officielle est un axe de travail que le SSP Lab mène plus largement, sur d'autres nomenclatures de la statistique publique — notamment la codification automatique en nomenclature **NACE** (activité économique) et en **PCS 2020** (professions et catégories socioprofessionnelles).

2 Les modèles de classification automatique actuels

En amont de ce stage, le SSP Lab a développé un pipeline de codification automatique pour la BdF, documenté en détail sur insee.fr/lab.github.io/codif-coicop-bdf. Il enchaîne un prétraitement, une étape de règles, quatre classifieurs indépendants, puis un arbitrage final :

Prétraitement → Regex → 4 classifieurs en parallèle (LCS, RAG-notices, RAG-annotations, TTC) → arbitrage par un LLM-as-a-judge → sortie finale

2.1 Prétraitement et règles (regex)

Le **prétraitement** met en forme les libellés bruts (nettoyage, structure) et construit les jeux d'annotations utilisés pour entraîner et évaluer les modèles. Les libellés ainsi normalisés passent ensuite par une étape de **règles regex**, qui code directement les entrées très standardisées (motifs ancrés du type $\hat{\dots}$) sans recourir à un modèle. Cette étape ne couvre qu'une faible part des libellés — la plupart des entrées réelles, trop hétérogènes, lui échappent et sont transmises aux quatre classifieurs.

2.2 Les quatre classifieurs de base

- **LCS** (*Longest Common Subsequence*) code par similarité de chaînes de caractères : il compare le libellé à coder à un référentiel de libellés déjà codés et retient le code du plus proche au sens de la plus longue sous-séquence commune.
- **RAG sur notices** effectue une recherche sémantique (base vectorielle Qdrant) dans les notices officielles de la nomenclature COICOP, et en déduit un code à partir de la notice la plus proche du libellé.
- **RAG sur annotations** applique le même principe de recherche sémantique, mais sur un corpus de libellés déjà codés par des gestionnaires (apprentissage par exemples similaires, *few-shot*) plutôt que sur les notices officielles.
- **TTC** (*TorchTextClassifier*) est un classifieur neuronal, entraîné sur des données de tickets de caisse puis affiné (*fine-tuné*) sur les données 2024 de la BdF, qui prédit directement un code au niveau 4 de la nomenclature.

Ces quatre classifieurs tournent en parallèle et produisent chacun une proposition de code, éventuellement assortie d'un score de confiance.

2.3 L'arbitrage final par LLM-as-a-judge

Lorsque les classifieurs ne sont pas unanimes, un grand modèle de langage est sollicité en dernier recours : il reçoit les propositions concurrentes des quatre classifieurs et tranche pour ne retenir qu'un seul code final. Ce jugement est le maillon le plus fiable de la chaîne — c'est aussi le plus coûteux à faire tourner sur l'ensemble des observations de l'enquête, en temps comme en ressources, et le moins interprétable : ses scores de confiance sont parfois difficiles à expliquer.

C'est précisément cette étape que ce stage visait à interroger : proposer une alternative moins coûteuse au LLM-as-a-judge, à ne solliciter que lorsque c'est réellement nécessaire, en s'appuyant sur des modèles plus légers et plus interprétables. La section suivante retrace la démarche suivie pour y parvenir, les différentes pistes explorées et les résultats obtenus.

3 Démarche et avancées du stage

Les premières semaines du stage ont été consacrées à la prise en main du pipeline existant, puis à une analyse descriptive systématique de l'accord entre les quatre classifieurs de base, sur 5 825 observations annotées du run de référence. Cette analyse poursuivait un double objectif : dresser un état des lieux fiable de la performance en amont de toute modélisation, et identifier des situations où l'on peut se passer du LLM-judge par une simple règle de bon sens.

TABLEAU 1

Niveau COICOP	Accord unanime des 4 classifieurs	dont corrects
1	59.7%	99.5%
2	55.1%	99.5%
3	37.9%	99.0%
4	30.3%	99.2%

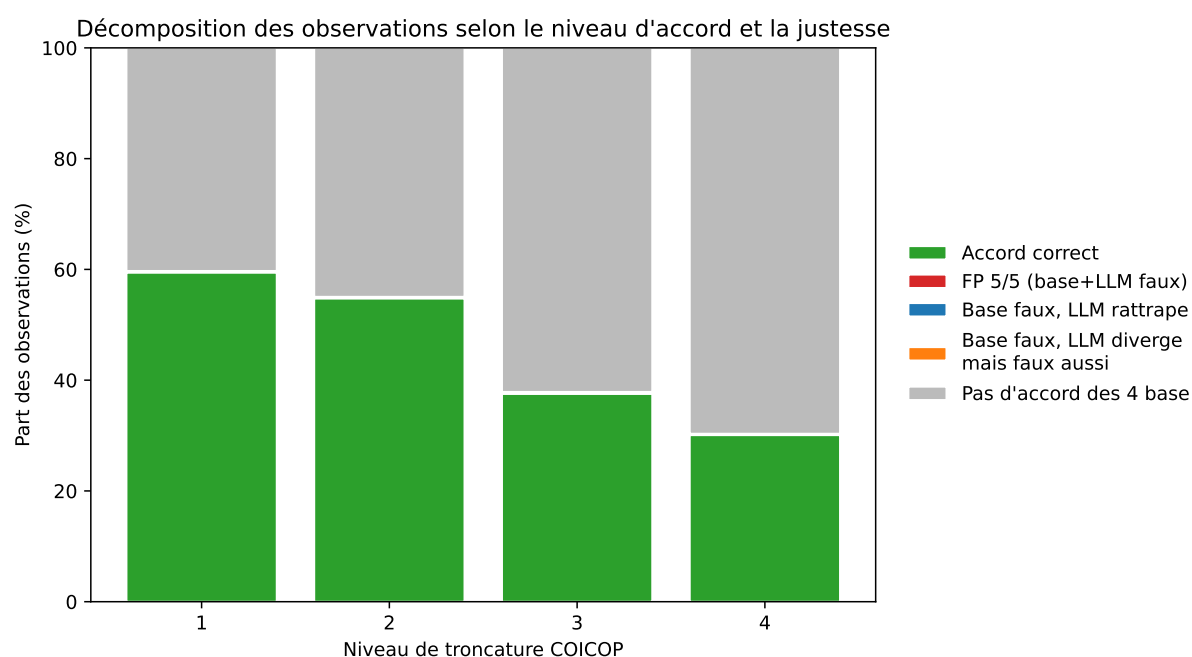


FIGURE 3.1 – Décomposition des observations selon le niveau d'accord et la justesse, par niveau de troncature COICOP

Le même constat se lit visuellement sur la figure ci-dessus : plus le niveau de détail exigé est fin (de la division au niveau 4), moins l'accord unanime est fréquent, mais sa fiabilité reste quasiment inchangée — l'essentiel du volume restant se partage entre désaccords partiels (à trancher par consensus majoritaire ou LLM) et une part résiduelle de faux positifs partagés, y compris par le LLM-judge.

Accord unanime. Selon le niveau de granularité retenu, entre 30 % (niveau 4, sous-classe) et 60 % (niveau 1, division) des observations voient les quatre classifieurs converger vers le même code. Dans ce cas, le consensus est presque toujours le bon (plus de 99 % de justesse, à tous les niveaux) : l'accord unanime est un signal de confiance très fiable.

Trois classifieurs contre un dissident. Lorsque trois classifieurs sur quatre s'accordent

contre un isolé (27.3% des cas au niveau 4), la majorité l'emporte à raison dans 91.5% des cas ; le dissident n'a raison que 6.6% du temps. Suivre la majorité reste donc, là aussi, une règle globalement fiable — même si le risque d'erreur n'est plus négligeable.

Un seul classifieur a raison. Dans 15.3% des cas, un seul classifieur trouve le bon code contre les trois autres : ce sont les cas les plus difficiles, ceux où l'arbitrage du LLM-judge apporte le plus de valeur ajoutée.

	Classifieur	Effectif	Part des cas
0	RAG-ANN	843	94.4%
1	TTC	36	4.0%
2	LCS	9	1.0%
3	RAG	5	0.6%

Ce classifieur « sauveur » est presque toujours RAG-ANN, loin devant TTC, LCS et RAG : la recherche par similarité sur des libellés déjà annotés par des gestionnaires s'avère, sur ces cas difficiles, nettement plus précieuse que les notices officielles ou le classifieur neuronal seuls.

3.1 Une baseline low-cost : vote majoritaire et arbitrage TTC

Sur cette base, le premier classifieur du stage a été mis en place : une **baseline** qui retient le code majoritaire parmi les quatre classifieurs et bascule vers le code proposé par TTC — l'arbitre le moins coûteux de la chaîne — en cas d'égalité ou d'absence de vote. Cette règle simple généralise l'observation précédente : suivre le consensus quand il existe, et s'appuyer sur un arbitre par défaut sinon.

TABLEAU 2

	Accuracy (niveau 4)
Modèle	
TTC seul	74.3%
Baseline (vote majoritaire + TTC)	81.4%
LLM-as-a-judge	81.9%

Cette règle, pourtant élémentaire, atteint 81.4% d'accuracy au niveau 4 — à comparer aux 74.3% de TTC utilisé seul (soit +7.1 points grâce au seul effet du vote majoritaire) et aux 81.9% du LLM-judge : l'écart avec l'arbitre le plus coûteux de la chaîne n'est que de 0.5 point. Cette proximité de performance a validé, dès cette étape du stage, l'hypothèse de départ : une bonne partie du travail d'arbitrage du LLM-judge peut être reproduite à moindre coût par une règle simple. Restait à savoir *sur quels cas* lui faire confiance sans risque excessif — l'objet de la modélisation suivante.

3.2 Détecter les erreurs de la baseline par Random Forest

La baseline seule ne suffit pas : encore faut-il savoir, au cas par cas, si l'on peut lui faire confiance ou s'il faut basculer vers le LLM-judge. L'étape suivante du stage a donc consisté à entraîner un modèle supplémentaire, chargé de prédire — sans jamais appeler le LLM — si la baseline a trouvé le bon code.

Principe d'une Random Forest. Une forêt aléatoire est un ensemble de nombreux arbres de décision, chacun entraîné sur un tirage aléatoire des observations (*bagging*) et d'un sous-ensemble aléatoire des variables à chaque nœud ; la prédiction finale agrège le vote de tous les arbres. Cette double source d'aléa décorrèle les erreurs individuelles des arbres et rend le modèle plus robuste qu'un arbre unique, tout en restant capable de produire une probabilité par observation et une **importance** pour chaque variable (mesurée par la réduction moyenne d'impureté qu'elle apporte dans les arbres).

La cible du modèle est binaire — *la baseline a-t-elle trouvé le bon code au niveau 4 ?* — et les variables explicatives excluent volontairement tout ce qui viendrait du LLM-judge, puisque l'objectif est justement de s'en passer : codes prédits par chaque classifieur (tronqués au niveau division), taille du plus gros groupe d'accord entre eux (`nb_accords_max`), unanimité, scores de confiance individuels (LCS, RAG, RAG-ANN, TTC), budget de la dépense, type d'enseigne et source de l'observation.

	Accuracy	ROC AUC
Évaluation		
Split train/test (20 %)	91.2%	0.958
Validation croisée (5 folds)	89.0% $\pm$ 0.7%	0.955 $\pm$ 0.005

Le modèle atteint 91.2% d'accuracy sur le test (ROC AUC 0.958), contre 89.0% $\pm$ 0.7% en validation croisée : l'écart entre les deux est faible, le résultat n'est donc pas un simple effet du découpage train/test. Un tuning des hyperparamètres par `RandomizedSearchCV` a également été testé ; il n'améliore pas le rappel sur les vraies erreurs de baseline au seuil par défaut, et le modèle par défaut (`n_estimators=300`, `min_samples_leaf=5`, `class_weight="balanced"`) a donc été retenu pour la suite.

TABLEAU 3

	Importance (%)
Variable	
<code>nb_accords_max</code>	35.6
<code>ttc_conf_1</code>	17.3
<code>unanimité</code>	7.9
<code>rag_confidence</code>	7.3

TABLEAU 3

Variable	Importance (%)
ragann_code_n1	5.5

Sans surprise, le signal principal du modèle est le niveau de consensus entre classifieurs de base (**nb_accords_max** domine largement les importances), suivi de la confiance de TTC et de l'unanimité; les scores de confiance individuels des autres classifieurs et le budget pèsent chacun nettement moins.

3.3 Analyse des erreurs manquées et compromis précision/rappel

L'accuracy globale masque une question plus opérationnelle : parmi les vraies erreurs de la baseline, combien le modèle laisse-t-il passer à tort — en leur attribuant $P(\text{baseline correcte}) > 0.5$ alors qu'elle a bel et bien faux ? C'est cette catégorie de faux négatifs qui est la plus coûteuse, puisqu'elle ne sera jamais envoyée au LLM-judge.

Sur les 217 vraies erreurs de baseline du test (18.6% des observations), le modèle en repère 86.6% (rappel), mais en manque encore 29. L'examen de ces cas manqués (comparés à ceux bien détectés) fait ressortir un profil commun : un signal de confiance élevé — accord entre classifieurs et/ou confiance de TTC — qui s'avère malgré tout faux. La confiance de TTC est en particulier le signal le plus trompeur : sa médiane est nettement plus haute sur les cas manqués que sur les erreurs bien détectées, ce qui contamine la probabilité prédite par la RF, cohérente avec le fait que **ttc_conf_1** soit la deuxième variable la plus importante du modèle. Le modèle n'a, en l'état, aucune information capable de distinguer un consensus fiable d'un consensus biaisé; améliorer sa détection nécessiterait une variable capturant *pourquoi* les classifieurs sont d'accord (par exemple la proximité lexicale avec des cas ambigus déjà connus), plutôt qu'un simple comptage d'accords.

Cette probabilité $P(\text{baseline correcte})$ peut cependant être seuillée : en faisant varier le seuil à partir duquel on décide d'envoyer une observation au LLM plutôt que de faire confiance à la baseline, on arbitre entre le nombre d'erreurs manquées et le volume — donc le coût — envoyé au LLM.

TABLEAU 4

Seuil	% volume envoyé au LLM	% erreurs de baseline détectées
0.99	83.9%	100.0%
0.95	68.4%	100.0%
0.90	57.9%	99.1%

TABLEAU 4

Seuil	% volume envoyé au LLM	% erreurs de baseline détectées
0.80	44.2%	96.8%
0.50	22.4%	86.6%

Un seuil à 0.90, par exemple, ne manque plus qu’une poignée d’erreurs (rappel proche de 100 %) en envoyant environ 60 % du volume au LLM — soit encore 40 % d’appels évités par rapport à un envoi systématique — tandis que le seuil par défaut (0.5) reste le plus économique mais laisse passer davantage d’erreurs non détectées. Ce compromis, plutôt qu’un seuil unique « optimal », est le véritable levier opérationnel : il permet d’arbitrer entre le coût du LLM et le risque toléré d’erreur silencieuse, plutôt que de figer une réponse binaire à la question.

3.4 Robustesse : généralisation à un nouveau run et correctif de convention

Le modèle validé sur le run de référence n’a de valeur opérationnelle que s’il tient sur des données futures. Il a donc été appliqué, sans être réentraîné, à des runs plus récents de l’enquête (juillet 2026) — un test de robustesse face au *drift* naturel des données (nouvelles observations, distribution de dépenses différente).

Ce test a d’abord révélé un écart de performance inattendu et trop marqué pour être imputable au seul *drift*. L’investigation a exclu les explications les plus courantes (fuite train/test via des libellés dupliqués, différences de composition entre runs sur la source ou le budget) et a fini par identifier une véritable convention manquante dans la comparaison des codes : en nomenclature COICOP, un code se terminant par « 0 » au niveau le plus fin (ex. 13.9.0) signifie *pas de sous-catégorie plus précise*, et doit donc être considéré équivalent au code tronqué d’un niveau (13.9). Or la fonction de comparaison utilisée jusque-là traitait ce cas comme une erreur — pénalisant à tort tout classifieur qui s’arrêtait, à raison, un niveau plus tôt.

La correction de cette convention (troncature des « .0 » terminaux en cascade dans `tronquer_niveau`) a mécaniquement amélioré les résultats sur l’ensemble du projet, avec un effet particulièrement marqué sur le run le plus récent :

Indicateur (niveau 4)	Run de référence (29/06)	Run plus récent (23/07)
Accuracy RAG-ANN	+13,2 points	+9,8 points
Accuracy baseline (vote majoritaire)	+4,0 points	+11,0 points

Indicateur (niveau 4)	Run de référence (29/06)	Run plus récent (23/07)
Accuracy du modèle RF (test 20 %)	+5,2 points	—
ROC AUC du modèle RF	+0,012	+0,112

Après ce correctif, un écart résiduel subsiste néanmoins entre runs : le modèle RF entraîné sur le run de référence, appliqué tel quel à un run d'août 2026 (2026-07-30), tombe de 91,2 % à 76,2 % d'accuracy. Réentraîner le même modèle directement sur ce nouveau run ne récupère qu'une partie de l'écart (80,4 %) : le nouveau run est donc, dans une certaine mesure, intrinsèquement plus difficile à prédire — pas seulement un problème de généralisation du modèle — ce qui reste une question ouverte pour la suite du projet (cf. conclusion).

3.5 Une piste interprétable : SIRUS

En parallèle du modèle Random Forest, une piste alternative a été explorée avec **SIRUS** (*Stable and Interpretable Rule Set*) : une méthode qui extrait, à partir d'une forêt aléatoire, un petit ensemble de règles logiques stables (typiquement une dizaine), au lieu de faire agir la forêt comme une boîte noire. Chaque prédiction devient alors directement justifiable par une poignée de règles explicites (par exemple « *si nb_accords_max >= 3 et ttc_conf_1 >= 0.9 alors baseline correcte* »), au prix d'un pouvoir prédictif généralement un peu inférieur à celui de la forêt complète dont les règles sont extraites.

Entraîné sur les mêmes variables que la Random Forest, SIRUS a été comparé à cette dernière sur le sous-ensemble le plus incertain des observations (probabilité de confiance de SIRUS inférieure à 0.6), c'est-à-dire précisément les cas où un modèle plus fin est le plus utile, ainsi qu'avec et sans les variables `budget`, `shop_type_name` et `source`.

À compléter : les résultats chiffrés de cette comparaison (`notebooks/synthese_sirus.qmd`, `notebooks/sirus_rapport.qmd`) sont en R et nécessitent le package `sirus` pour être recalculés ; ils n'ont pas pu être régénérés dans cet environnement de rédaction (R non installé) et seront intégrés dans une prochaine passe.

Cette piste reste documentée comme alternative interprétable si le compromis performance/explicabilité devait un jour pencher en faveur de la transparence plutôt que de la seule accuracy.

3.6 Bilan comparatif des approches

Le tableau suivant synthétise, sur le run de référence, l'accuracy au niveau 4 des différentes approches successives du stage :

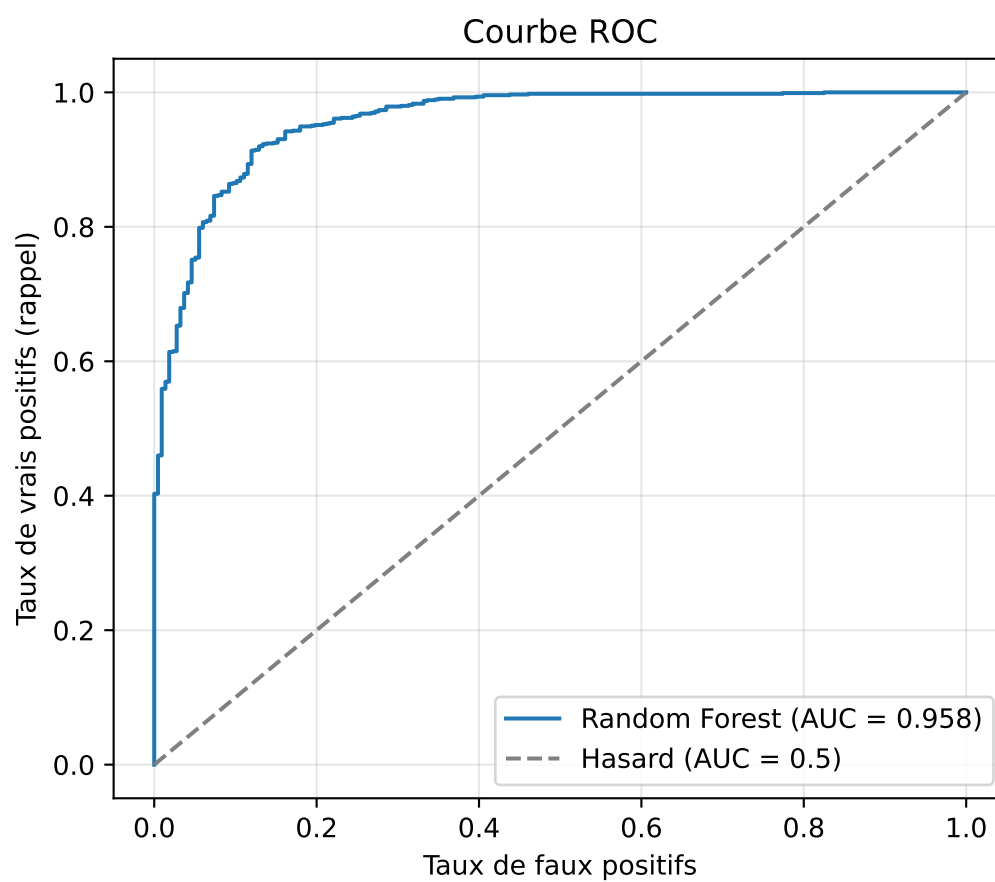


FIGURE 3.2 – Courbe ROC du modèle Random Forest

Approche	Accuracy (niveau 4)
TTC seul	74,3 %
Baseline (vote majoritaire + TTC)	81,4 %
Baseline + Random Forest (filtrage au seuil 0.90)	99,1 % des erreurs détectées, pour 57,9 % du volume envoyé au LLM
LLM-as-a-judge (référence, coûteux)	81,9 %

Le classifieur simple (baseline) approche déjà la performance du LLM-judge à moindre coût ; couplé à la Random Forest pour trier les cas incertains, il permet de n'envoyer au LLM qu'une fraction du volume total, tout en détectant la quasi-totalité de ses propres erreurs. C'est ce compromis — plutôt qu'un modèle unique « meilleur que le LLM » — qui constitue le principal résultat opérationnel du stage.

4 Conclusion et perspectives

Ce stage visait à proposer une alternative moins coûteuse au LLM-as-a-judge pour la codification automatique COICOP de l'enquête Budget de Famille, à ne solliciter que lorsque c'est réellement nécessaire. La démarche suivie a consisté à avancer par étapes successives, chacune validée avant de passer à la suivante : une analyse descriptive de l'accord entre les quatre classifieurs de base pour identifier où leur consensus est fiable ; une baseline low-cost (vote majoritaire, arbitrage TTC) qui approche déjà, à elle seule, la performance du LLM-judge ; une Random Forest capable de prédire si cette baseline a raison, avec un seuil de décision ajustable selon le compromis coût/risque recherché ; enfin, une vérification de la robustesse de l'ensemble sur des runs plus récents, qui a mis au jour et corrigé une convention de comparaison manquante dans la nomenclature COICOP, avec un effet mesurable sur l'ensemble des résultats du projet.

Ce dernier point illustre une leçon plus générale du stage : une part significative du travail n'a pas consisté à améliorer des modèles, mais à fiabiliser la mesure de leur performance elle-même — sans quoi toute comparaison entre approches, ou tout diagnostic de dégradation dans le temps, reste fragile.

Plusieurs pistes restent ouvertes pour la suite du projet :

- **Réduire les faux négatifs de la Random Forest**, c'est-à-dire les cas où elle fait à tort confiance à la baseline : le compromis précision/rappel documenté plus haut donne un levier de pilotage, mais améliorer la détection elle-même demanderait une variable capturant *pourquoi* les classifieurs sont d'accord (proximité lexicale avec des cas ambigus connus), plutôt qu'un simple comptage d'accords.
- **Comprendre l'écart de difficulté intrinsèque entre runs**, mis en évidence après correction de la convention de troncature : réentraîner le modèle sur un run

récent ne suffit pas à retrouver la performance du run de référence, ce qui suggère un facteur encore non identifié plutôt qu'un simple problème de généralisation.

- **Finaliser la comparaison avec SIRUS**, laissée en suspens dans ce rapport faute d'environnement R disponible au moment de la rédaction, pour statuer sur l'intérêt d'un modèle interprétable par construction face à la Random Forest.
- **Poursuivre l'alimentation du site de suivi** en statistiques descriptives et en visualisations, au fil des nouveaux runs de l'enquête, pour que les résultats présentés ici restent reproductibles et à jour.

Au terme de ce stage, le pipeline dispose ainsi d'une alternative opérationnelle et documentée au LLM-as-a-judge systématique : un classifieur simple et gratuit, complété d'un filtre statistique qui décide, au cas par cas, quand ce classifieur peut être suivi tel quel et quand l'arbitrage plus coûteux du LLM reste nécessaire.